nature.com [[light_sheet_microscopy/lemon_keller_2015/paper|paper]]

[[Pasted image 20250701141251.png]] # Whole-central nervous system functional imaging larval Drosophila ## Key takeaways: - Whole CNS - Brain + Spinal Cord - Effectively measured full CNS in first 2 instar periods - Only a large fraction of 3rd instar period due to thicker tissue - First study to show regional, gradients of coordinated activity (see supp. videos 1-4) - Foreword motions show activity in SOG (ventral), backwards motions show activity in dorsal areas of the CNS - SOG may initiate movement, dorsal brain regions terminate movement - Population Analysis - Individual neurons also show periodic bouts of activity matching behavior waves - 2 -5 Hz (2 photon / 1 photon respectively) - 20,000 volumes/sample - 5 volumes / second / camera - 370 images per second - Spatial alignment of 4 focal planes - Capability: [400 x 200 x 200]um^3

3 challenges to overcome:

1. Microscope
    1. Multi-view: 25-fold better temporal resolution
    2. Move the sheet, not the sample
2. Specamin prep
    1. Up to an hour in non-transparent tissue
3. Computational pipeline
    1. Several TB/experiment
    2. Map functional activity across CNS during specific behaviors ## Fig 2: Whole CNS Functional Imaging

[[Pasted image 20250703103849.png]]

- Imaging at 5 Hz, yet "constant imaging speed of 370 fps"
- So, full 3D volume @ 5 fps
- Each volume ~75 z-planes
- 72 zplanes in a volume, and do that 5 times per second, thus 5-hz
- 72 * 5 = 370fps

Supp. Figure 1: Multi-view coverage

[[Pasted image 20250701160830.png]]

- Images taken by each camera for each light sheet (CamA = dorsal, CamB = Ventral)
- Cameras have a complimentary light sheet
- Max Projections (top), single z-planes (bottom)
- Illustrates the variable quality between cameras
- Shows nuclear labeled nuclei

Supp. Figure 2: Theoretically optimal light sheet config

[[Pasted image 20250703112153.png]]

- Full-width-half-maximum: Measurement of resolution, smaller = sharper images
- A: Experimental measurement of how the beam thickens as you move away from the center
  - A single light sheet produces good resolution only in the center when the sheet is sub ~3um?
- B: Now with duel sheets, whichever sheet is more thin dominates the signal
  - Utilize both sheets to keep thickness low

Fig 3. Fictive locomotion in CNS

[[41467_2015_Article_BFncomms8924_Fig3_HTML.jpg]]

- To assess how higher order structures (brain lobe) coordinate with motor centers (thorax / abdomen), analyze **activity timing relationships** between these structures
- Is there synchrony between brain/belly that is *specific to a behavior*
- Movement foreward / backward is synched between
- They are not yet looking at behaviors, just quantifying responses. Behavioral analysis is Fig 5.
- First 120s of activity ignored (non-stationary signal drift)
- Each trace z-scored (F - mean / std)
- Subtract mean response across all segments
  - Highlight relative changes between segments
- Average left/right hemisegments
- Embed in SVD / PCA
  - Amplitude and Phase in 2D space correspond to presence and direction of waves

Fig 4. Mapping entire CNS during behaviors

[[41467_2015_Article_BFncomms8924_Fig4_HTML.jpg]]

- A. data from a single voxel. This is done independently for the entire volume.
  - Motor neurons are primarily located in dorsal (BL) regions, they should be faster and more prevelent
- Indeed they are (B)
- Measure when / what extent peaks are in sync with waves
- Traces fit using information about timecourse, vs without this information
- How well traces fit a rectangular function with fixed radius r

Fig 5. Profile single-neuron during behaviors

[[Pasted image 20250703114626.png]]

- Projection of manually segmented soma (a, n=200), and activity averages across all wave time windows (b) -8 to 8s around locomotive behavior

Fig 6: Compare CNS to spinal cord via Nonlinear Image Registration to create a reference template.

[[**Pasted image 20250703115027.png**]]

- 6 independent CNS preps to assess / correct variability between specimen
- Get mean-intensity projections over time (Reference Volume)
- Manually mask nerves so it doesn't contribute to registration
- Two step: Affine, iteratively deformable registration
  - Minimize the deformation for any individual CNS prep
- B-spline diffeomorphic image registration
  - Gradient Step 0.1
  - Five subsampling factors (10, 6, 4, 2, full) [px]
  - Cross-correlation with neighbor radius of 6
- Quality assessed via manually annotating 3D landmarks and measuring the distance they moved during registration ### Supp. Fig 4, 5, 6: Spatial Maps for each instar period

1st instar [[**Pasted image 20250701161912.png|800**]] [[**Pasted image 20250701161939.png|800**]] [[**supp_movie_6.mov**]]

vs 2nd instar [[**Pasted image 20250701164046.png**]]

[[supp_movie_7.mov]]

These are acquired *after* the functional dual light-sheet imaging,(optimizes for speed), with a single light sheet for highest resolution. 1. After fusing the images, a blob detector gets the centroid positions. 1. Effectively segmentation 2. Very conservative (minimize false +) 3. This is used as a local "seed" 2. Search for the brightest local pixel 1. Not correlated pixels, i.e. suite2p 2. In a cube, find brightest pixel, accounts for anisotropy/duplicates 3. For each XYZ nucleus, take an intensity profile (cross section) 1. Modeled as a gaussian 2. Sharper, narrow gaussian indicates higher resolution 3. std, how spread the brightness is 4. median of the sigmas (x+ x- y+ y-) gives spread in the plane 5. mean of z+ z- gives axial resolution 6. Convert to full-width-half-maximum 4. Remove ~0.38 um resolution of artifact from light scattering

Fig 7: Multi-specimen neurons spatial locations

[[**Pasted image 20250703120407.png**]]

- 3 spatially, non-affine registered CNS
- B shows close spatial relationship of soma's in different samples
- Colors are specimen identity

[[supp_movie_5.mov]] [[supp_movie_6.mov]] [[supp_movie_7.mov]] [[supp_movie_9.mov]]

Data Strategy:

- Duel channel: functional (GCaMP6s) , and a nuclear reference (tdTomato)
- Yields two-channel, multi-view image stacks
  - (~40 images per volume across 2–3 cameras)
  - ~1.0 m lateral / 2.5 m axial resolution in first-instar larvae, and
  - ~1.6 m / 5.9 m in third-instar larvae (larger, more scattering tissue)
  - Resolution higher in the ventral nerve cord than in the brain lobes due to shorter light paths in the cord

Compute Overview

1. Compress, Use 3D geometry to separate foreground / background
    1. Discard background, compress foreground
    2. sCMOS dead-pixel correction
2. Multi-view fusion: [Chan -> Chan, Camera -> Camera]
    1. Align foreground image-stacks between cameras (rigid)
    2. Blend images using the geometric model
        1. Minimize the distance each photon travels
3. Temporal registration (Correct for specimen drift)
    1. Register each time-point to the midpoint of the recording
    2. No specifics described here
4. Compute DF/F for every voxel
    1. 25th/10th (1p/2p) percentile intensity level
    2. 70-timepoint sliding window
5. Detect and classify which behavior the animal is producing
6. Map activity in different brain regions to specific behaviors

Deep Code Analysis

Data Properties

```
`2048 x 752 pixels`
`79 z-planes`
`5.13 um z-spacing`
`1 Hz volumetric imaging (Sequential mode)`
- 2 channels, 3 cameras / timepoint
```

- metadata (dims, exposure time,) saved as XML
- .klb, .jp2, .tif
- SPMXX/TMYYYYYY/…
- **Processing Modules** (in order of use)
    1. clusterPT.m (processTimepoint.m)
    2. clusterMF.m (multiFuse.m)
    3. localAP.m (analyzeParameters.m)

4. clusterTF.m (timeFuse.m)
5. clusterRS.m (registerStacks.m)
6. localCR.m (clusterCD.m)
7. clusterCD.m (calculateDelta.m)
8. detectWaves.m
9. computeInformationGain.m
    1. localCP (collectProjections.m)
    2. convertData
10. Compression/decompression
11. Cropping, missing timepoints, bg percentile values
12. read/write tiff stacks

1. Data management
2. Multi-view image fusion
    1. clusterMF.m (multi-fusion)
        1. Input dual-camera raw
        2. Fuses camera inputs (2 cameras up to 4?) -> 3D stack per timepoint
        3. Input: `SPM*/TM*/…klb`
        4. Output: 3D stack for each timepoint `…/multiFused/…`
        5. What it does
            1. Register cam N to N + 1 (gradient decent)
            2. Split foreground / background

- Default percentile is 5% (lowest 5% signal is background)

    2. clusterTF.m (timepoint-fusion)
    3. Match brightness levels
    4. collectProjections.m
        1. xy, xz, yz

3. Functional Data Processing

- `multiFuse.m` is run first, for all timepoints independently
- `timeFuse.m` is run afterward to **align each fused volume to a reference timepoint**, using stored transformation offsets and masks ### multiFuse.m
- modes: 4-view, 2-view camera, 2-view channel
    - Channels are td-tomato + GCaMP6s
- 1. Align Channels (alignChannels.m)
    - Mask background out of foreground
        * Preprocess: spatial crop + gaussian filter
        * get min intensity from min() or percentile()
        * foreground mask
            · threshold=min+maskFactor (mean−min)
    - Align slices across cameras

    - Take XZ slice
  - Channel registratioin
    - fun = @(x) transformChannel(…);
    - [xSol, …] = fminuncFA(…);
    - save `.transformation.mat`

- 2: Fuse Channels (fuseChannels.m)
  - Load X-Z slices `*.xzMask`, `*.mask2D`
  - Apply the rigid transformation saved previously
    - Affine transform with rotation + z-offset
  - Check for / re-apply the mask/Gaussian filtering in step 1
  - Normalize Channels: compute a correction factor that will match the dynamic range of the two channels
  - Blending
    - If fusionType=0/1 (linear blending)
      - For each pixel, which channel dominates the mask, choose that pixel?
      - `fused = chan1 * weight1 + chan2 * weight2` ‘
    - if fusionType = 2: Wavelet Function
      - `wfusimg(…, 'db4', 5, 'mean', 'max')`
    - if fusionType = 3: Average
    - `fused = (stack1 + stack2) / 2`
  - Save fused stacks

- 3: Camera alignment (alignCameras.m)
  - Gaussian smooth, mask background
  - Load substack, normalize intensities
  - Iteratively shift and score normalized cross-correlation
    - `scores(x, y, 3) = sum(data1 .* shifted_data2) / (sum1 * sum2)`
  - X shift, Y shift, Z rotation
    - Z-rotation accounts for camera mounting?

- 4: Fuse Cameras (fuseCameras.m)
  - Compute average mask
    - maskFusion=0: overlap, intersection of both masks
    - maskFusion=1: union of only valid regions
  - (optional) Intensity correction
  - if fusionType = 0/1:
    - weight = [0 → 0.5] for fading camera
    - weight = [1 → 0.5] for dominant camera
  - if fusionType = 2: wavelet per z-plane
  - if fusionType = 3: average both stacks at each voxel
  - `.fusedStack`: the full fused 3D volume
  - `*_xyProjection`, `*_xzProjection`, `*_yzProjection`: max projections

Inputs:

`parameterDatabase` or `*.mat` with: camera IDs, channels, masks, stack paths, correction params

- `*.xyMask`, `*.transformedMask3D`, `*.transformedMask2D`: 2D/3D masks computed during alignment (from `alignCameras.m` or earlier)
- `*.minIntensity.mat`: for background estimation (if `dataType == 1`)

Intermediates:

File	Purpose
`*.transformation.mat`	Coarse-to-fine XY/rotation alignment between views
`*.fusionDataSlice`	Local fusion slabs used to estimate intensity correction
`*.stitchedStack`	Fused stack without blending (pre-blend version)
`*.transformedGauss`	Gaussian-filtered stack for mask creation
`*.transformedMask3D/2D`	Output of thresholding + cleanup for 3D segmentation

Outputs:

`*.fusedStack`	Final fused 3D volume
`*.fusedStack_xyProjection`	Max-Z XY projection
`*.fusedStack_xzProjection`	Max-Y XZ projection
`*.fusedStack_yzProjection`	Max-X YZ projection
`*.fusionMask`	`averageMask(x,y)`: Z-index of fusion per pixel
`*.correctedStack`	Background-corrected + intensity-scaled per-camera stacks
`*.intensityCorrection.mat`	Background values, intensity sums, correction factor

timeFuse.m

- Rigid alignment (rotation + translation)
- Intensity correction
- Channel fusion (within each camera)
- Camera fusion (between cameras)
- Adaptive spatial blending
- Optional wavelet or averaging fusion strategies
- Similar setup to `multiFuse.m` #### Inputs

Variable	Description
`parameterDatabase`	`.mat` file storing pipeline parameters
`t`	Time index into the `timepoints` array
`memoryEstimate`	Memory allocation hint used in processing

Intermediates

Variable	Description
`primaryDataArray`	Raw and processed image stacks (camera × channel)
`fusedStack`	Final fused output volume
`averageMask`	Adaptive mask for blending regions
`currentTransformations`	Transform/correction values from lookup table
`xSize, ySize, zSize`	Stack dimensions after cropping

Outputs

- `.fusedStack.klb` (main output)
- Intermediate stacks:
  - `.stack`, `.transformedStack`, `.correctedStack`, `.stitchedStack`
- Projection images (`XY`, `XZ`, `YZ`)
- `*.intensityCorrection.mat`
- `*.fusionDataSlice.klb`
- `_jobCompleted.txt` marker #### Reused in timeFuse:
- `.fusedStack` (used as reference source)
- `.fusionMask` (used for adaptive blending)
- `.intensityCorrection.mat` (may be reused or recomputed)
- `.correctedStack`, `.transformedStack` (used if not skipping steps) ## **Computational References**
- nonlinear (diffeomorphic) registration
  - Curved, nonrigid deformations
  - Models smooth warps
  - Local: individual voxels are deformed
  - Gaussian -> B-Spline basis function
  - ANTS: Github ## Questions / Other Information
- 2 Cameras main config, how prevelent is the use of 3 cameras
  - How much does this complicate downstream processing?
- multiFuse / timeFuse seem to be doing much of the same thing
- Maximum allowed volume: [800 x 800 x 250 um^3]
- Example: [1300 x 1300 x 1500 um^3] (at expense of frame rate)